

Kokkos Comm: Performance-Portable Communication

C. Nicole Avans & Evan D. Suggs

Agenda

Introduction

3

Getting Started

10

Live Demo

17

Summary

20



3

Introduction

Kokkos Comm is a performance-portability communication API. It exists to manage error-prone implementation details and provide a core interface for switching between communication backends (e.g., MPI, NCCL, RCCL, etc.)



**4**

Review of Major Concepts

Kokkos

Write-once, run-anywhere performance portability for on-node computing

**Kokkos
Comm**

Communication abstractions for hardware-agnostic off-node communication

MPI

Message Passing Interface, used generally to facilitate off-node communication

**NCCL /
RCCL**

Hardware-specific collective communication libraries for Nvidia and AMD GPUs, respectively

OpenMP

Parallel features for shared-memory environments

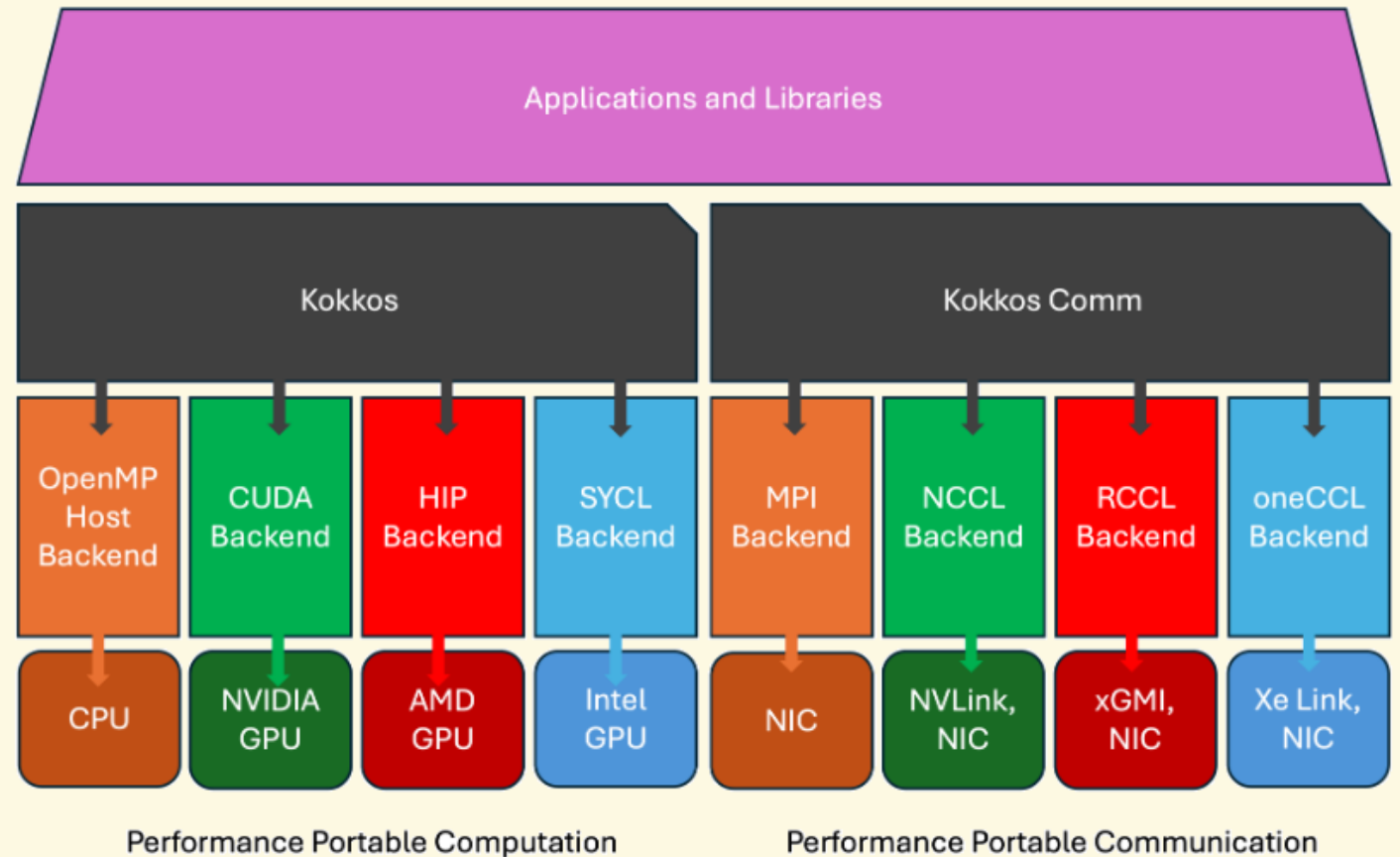


5

Kokkos Ecosystem Stack

Kokkos and Kokkos Comm exist within an ecosystem of libraries, and each tackle different facets of parallelism and performance-portability.

Question: How much, if any, hardware-specific C++ have you written at this point? Have you experienced any friction?





6

What's Error-Prone About It?

- If your data is non-contiguous, it must be marshalled (ordered), packed, sent, unpacked, then computed.
- MPI provides communication for a variety of hardware, but it deals with pointers rather than data structures.
- Communication has typically not been write-once run-anywhere: vendor-specific collective communication is written separately, and optimization requires knowledge about the architecture of the intended system.



7

Communication & Collectives

send / isend

recv / irecv

persistent
send / recv

reduce

← Point-to-Point →

broadcast

alltoall

allgather

allreduce

← Collective →

See the [MPI Standard](#) for exhaustive information about these and more communication patterns.

**8**

Non-blocking Send Comparison

MPI_Isend specification

```
int MPI_Isend(const void *buf,  
             int count,  
             MPI_Datatype datatype,  
             int dest,  
             int tag,  
             MPI_Comm comm,  
             MPI_Request *request);
```

KokkosComm::mpi::isend specification

```
template <KokkosExecutionSpace  
         ES, KokkosView SV>  
void isend(const ES &space,  
          const SV &sv,  
          int dst,  
          int tag,  
          MPI_Comm comm,  
          MPI_Request &req);
```

KokkosComm::send specification

```
template <KokkosExecutionSpace  
         ES, CommunicationSpace CS,  
         KokkosView SV>  
Request<CS> send(Handle<ES, CS>  
               &handle,  
               const SV &send_view,  
               int dest);
```

Reminder:

In C++, you can turn a pointer into a value with * (***buf**) and turn a value into a pointer with & (**&req**).



9

Persistent Communication

```
KokkosComm::Channel<> channel(dest_rank, src_rank, tag,  
MPI_COMM_WORLD);
```

```
Kokkos::View<Scalar*, Kokkos::HostSpace> send_host("send_host", N);  
Kokkos::View<Scalar*, Kokkos::HostSpace> recv_host("recv_host", N);
```

```
channel.sendinit(send_host);  
channel.recvinit(recv_host);
```

```
channel.start();  
channel.wait();
```

A simple persistent communication example.

An abstract graphic design on a light cream background. It features several thick, rounded lines in green, blue, and red. A green line starts from the left, curves down, and then continues horizontally. A blue line starts from the bottom, curves up, and then continues horizontally, overlapping the green line. A red line starts from the top right and curves down. There are two small black dots: one on the green line and one on the blue line. A large orange circle is positioned on the left side of the image.

Getting Started

**11**

Aside: Build Methods

CMake

- CMake is the de-facto standard for building C++ code.
- The `-S` flag specifies project root directory and `-B` flag specifies build directory.

```
sudo apt install cmake
```

Spack

- Spack is a package manager for making installing scientific software easy.
- Clone it from github, run the setup script and you're ready to go in any environment.

```
git clone https://github.com/spack/spack.git  
. spack/share/spack/setup-env.sh
```



Sources linked above.



12

Cluster Instructions Overview

- On clusters (or locally!) you may use spack to install Kokkos and proceed to installing Kokkos Comm with CMake.
 - This is similar to using `module load` on clusters.
- Alternatively, you can just use CMake for all of it.
 - It is a good idea to save your CMake commands to a build script (i.e., `[cluster_name]_build.sh`) so you can write it once and reuse it from there. Spack commands can also be put into a script. (An example will be shown later in this presentation.)

**13**

TN Tech Cluster

- Use `spack load kokkos@4.7~cuda gcc@14 openmpi cmake`
- You will also want to go to the directory `cd /work/classes/csc4760-001-2026s/<username>` and use your username.
- Additionally, use either `hpcshell --account=csc4760-001-2026s --cpus-per-task=2` or `salloc --account=csc4760-001-2026s --cpus-per-task=2` to go off the login node onto a node with actual resources.
- To get location of spack module (install directory), use `spack location -i <spack spec>` such as `spack location -i kokkos@4.7~cuda` to print the location
- Follow the rest of the instructions, but use cluster directories

**14**

Delta Cluster

- Use `module load cray-mpich cmake`
- Use `accounts` command to find what account you can use, should be `bfrm-delta-gpu`
- **Example:** `salloc --cpus-per-task=2 --gpus-per-node=1 --account=bfrm-delta-gpu --partition=gpuA40x4-interactive,gpuA100x4-interactive`

You will have to install Kokkos and Kokkos Comm yourself — more details to follow.

**15**

CMake Build Commands

Kokkos

```
cmake \  
  -S [source] \  
  -B [build] \  
  -DCMAKE_CXX_COMPILER=[compiler] \  
  -DCMAKE_INSTALL_PREFIX=[install] \  
  -DKokkos_ENABLE_[CUDA/HIP]=ON \  
  -DKokkos_ARCH_[type]=ON \  
  -DCMAKE_BUILD_TYPE=Release;
```

Kokkos Comm

```
cmake \  
  -S [source] \  
  -B [build] \  
  -DCMAKE_CXX_COMPILER=[compiler] \  
  -DKokkos_ROOT=[kokkos install];
```

Bracketed text indicates where specific information needs to be changed, such as providing **source**, **build**, and **install** directories, the location of the compiler, or where a flag needs to be modified to select an architecture and specify CUDA (Nvidia) or HIP (AMD).

**16**

WSL / Linux

```
#!/bin/bash
```

```
init() {
    printf "Setting up directories...\n"
    mkdir -p $HOME/class/repos/
    $HOME/class/install/kokkos
    printf "Complete!\n"
}

kokkos() {
    if [ -d $HOME/class/repos/kokkos ]; then
        printf "'kokkos' directory already exists.\n"
    else
        git clone git@github.com:kokkos/kokkos.git
        $HOME/class/repos/kokkos
    fi
    cmake \
        -S $HOME/class/repos/kokkos \
        -B $HOME/class/repos/kokkos/build \
        -
    DCMMAKE_INSTALL_PREFIX=$HOME/class/install/kokkos \
        -DCMAKE_BUILD_TYPE=Release \
        -DCMAKE_CXX_COMPILER=mpicxx \
        -DKOKKOS_USE_CXX_EXTENSIONS=ON;
    make -C $HOME/class/repos/kokkos/build -j16
    install
    printf "Kokkos setup complete!\n\n"
}
```

```
kokkos-comm(){
    if [ -d $HOME/class/repos/kokkos-comm ]; then
        printf "'kokkos-comm' directory already exists.\n"
    else
        git clone git@github.com:kokkos/kokkos-comm.git
        $HOME/class/repos/kokkos-comm
    fi
    mkdir -p $HOME/class/repos/kokkos-comm/build
    cmake \
        -S $HOME/class/repos/kokkos-comm \
        -B $HOME/class/repos/kokkos-comm/build \
        -DCMAKE_CXX_COMPILER=mpicxx \
        -DKokkos_ROOT=$HOME/class/install/kokkos \
        -DCMAKE_BUILD_TYPE=Release \
        -DKokkosComm_ENABLE_PERFTESTS=ON \
        -DKokkosComm_ENABLE_TESTS=ON;
    make -C $HOME/class/repos/kokkos-comm/build -j16
    printf "KokkosComm setup complete!\n\n"
}

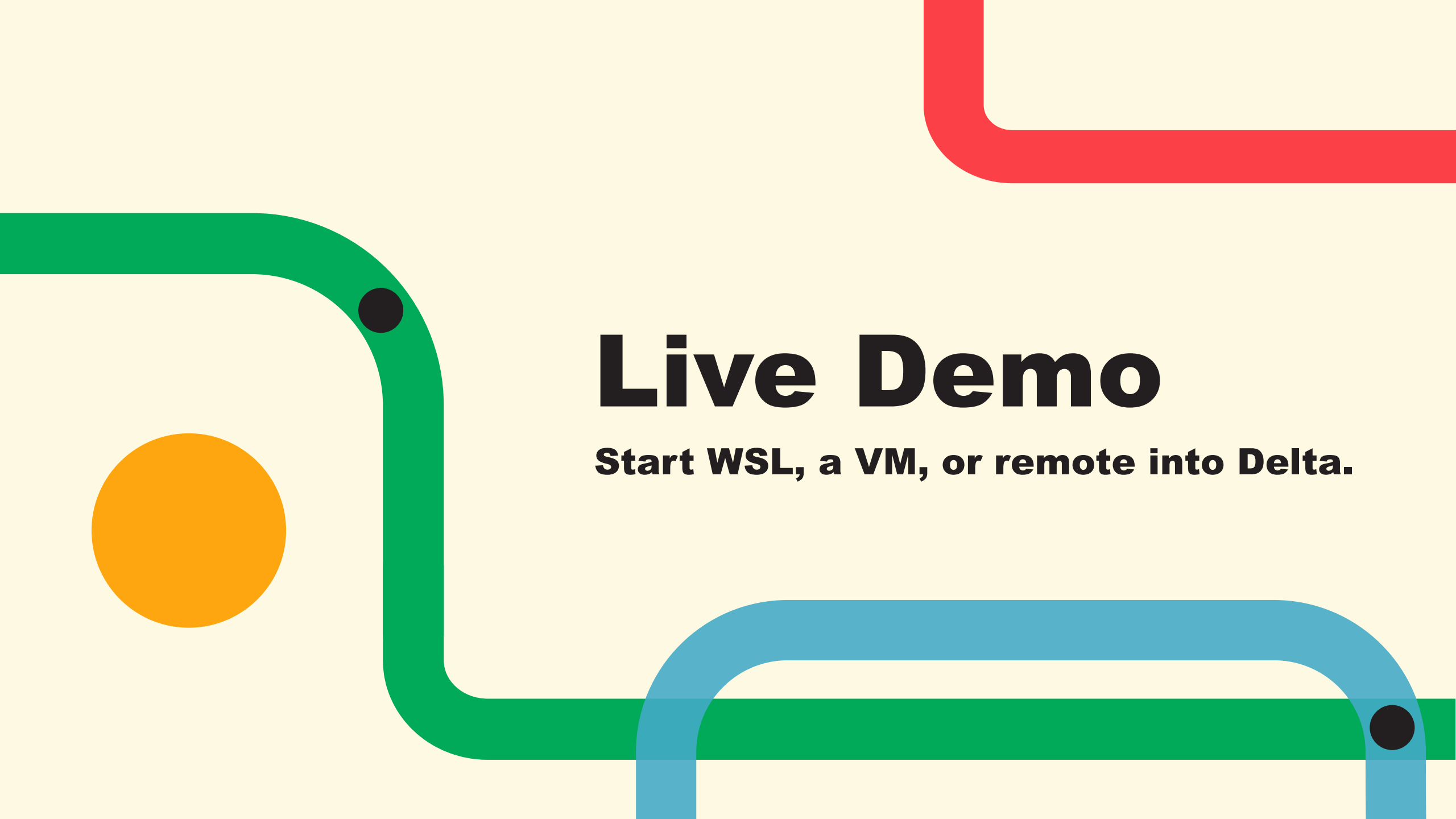
clean() {
    printf "Cleaning up...\n"
    rm -rf $HOME/class/install/kokkos $HOME/class/repos/kokkos-
comm/build
    printf "Cleanup complete!\n"
}

test(){
    cd $HOME/class/repos/kokkos-comm/build; ctest -V
}
```

```
# Check for arguments
if [ "$1" == "clean" ];
then
    clean
elif [ "$1" == "test" ];
then
    test
else
    set -e
    sudo apt install cmake
    sudo apt install
openmpi-bin libopenmpi-dev
    clean; init; kokkos;
kokkos-comm; test
fi
```

This is a complete, basic build script for WSL / Linux.

Question: Do you tend to use build scripts in your development? Is any of this unfamiliar?

An abstract graphic featuring a green line that starts from the left, curves down, and then continues horizontally. A blue line starts from the bottom, curves up, and then continues horizontally, overlapping the green line. A red line starts from the top right and curves down. An orange circle is positioned on the left side of the green line. Two black dots are located on the green line: one on the curve and one on the horizontal segment.

Live Demo

Start WSL, a VM, or remote into Delta.

Steps

Check for MPI

`which mpi`

Install Kokkos

Or, verify current install

Install Kokkos Comm

Or, verify current install

Run Tests

`ctest -v`

Create Experiment File

`touch example.cpp`

Don't worry – we can roll it back to previous slides to show commands again.



19

Aside: Testing

CMake provides a testing utility in the form of CTest. The command shown on the previous slide (`ctest - v`) runs tests with verbose output. Utilizing this function with installed scientific software (such as Kokkos and Kokkos Comm) will quickly illuminate any issues running on a given system.

In your own development, deploying software with tests built in assists your users and aids in maintaining the codebase.



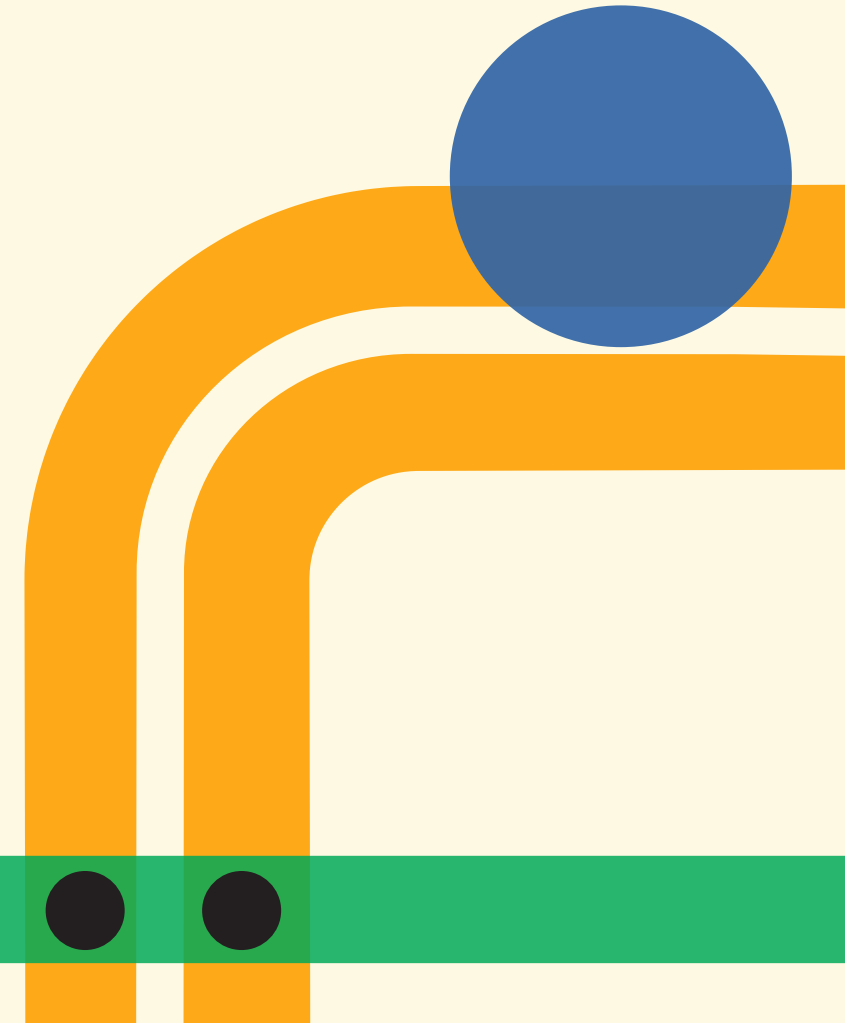


20

Summary

Kokkos Comm can make moving your data for parallel computation easier. In conjunction with **Kokkos**, it provides an enhanced experience for writing production code to be used in heterogeneous computing environments (like those found in academic, DOE laboratory, or industry clusters).

This allows you to focus on what you need to do with your data, rather than system-specific optimization or troubleshooting pointers.





21

Summary, cont.

This presentation intends to demonstrate basic setup and usage of Kokkos Comm within the Kokkos ecosystem, but is also an illustration of critical skills for working within scientific computing: **building**, **testing**, and **continued development** with integrated libraries. In particular, the use of build scripts is critical for reproducibility and collaboration.



22

Resource Review

- [Kokkos \(and its list of supported architectures\)](#)
- [Kokkos Comm](#)
- [MPI Standard](#)
- [CMake](#)
- [Spack \(and specific instructions for use with Kokkos\)](#)
- [CTest](#)

Feel free to follow up (contact info on next slide) for more information.

The background features abstract, thick, rounded lines in red and orange. A solid green circle is positioned in the lower right area. Two small black dots are placed on the red lines: one on a horizontal line on the left and another on a vertical line on the right.

Thank you, questions?

C. Nicole Avans

cnavans42@tntech.edu

Evan D. Suggs

esuggs@tntech.edu